



UNIVERSITÀ DEGLI STUDI  
DI SALERNO

CORSO DI LAUREA IN INFORMATICA

# Phishing Url Detection-AI

***Autore:***

Andrea Zeno

***Docenti:***

Prof. Polese Giuseppe  
Prof.ssa Caruccio Loredana

## Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione al problema</b>                                 | <b>4</b>  |
| 1.1      | L'evoluzione del Phishing                                       | 4         |
| 1.2      | I limiti degli Approcci Tradizionali                            | 4         |
| 1.3      | Obiettivo del Progetto: Approccio Proattivo con l'IA            | 4         |
| 1.4      | Il problema affrontato                                          | 5         |
| 1.4.1    | Il problema dell'explainability nei modelli di Machine Learning | 5         |
| 1.4.2    | L'importanza dell'Explainability nella Cybersecurity            | 6         |
| <b>2</b> | <b>Data Understanding and Data Preparation</b>                  | <b>7</b>  |
| 2.1      | Data Gathering                                                  | 7         |
| 2.2      | Verifica della Qualità dei Dati                                 | 7         |
| 2.3      | Data Cleaning                                                   | 7         |
| 2.4      | Feature Selection                                               | 8         |
| 2.5      | Analisi del Data Leakage                                        | 8         |
| 2.6      | Class Balance Analysis                                          | 9         |
| 2.7      | Analisi delle Correlazioni                                      | 10        |
| 2.8      | Preprocessing e Scaling                                         | 11        |
| 2.9      | Data Splitting                                                  | 11        |
| <b>3</b> | <b>Sviluppo dei Modelli di Classificazione</b>                  | <b>12</b> |
| 3.1      | Scelta degli Algoritmi                                          | 12        |
| 3.1.1    | Il Modello Baseline: Decision Tree                              | 12        |
| 3.1.2    | L'Evoluzione: Random Forest Classifier                          | 12        |
| 3.2      | Come valutare i modelli?                                        | 13        |
| 3.3      | Come migliorare l'explainability?                               | 13        |
| 3.4      | Implementazione                                                 | 13        |
| 3.5      | Analisi Critica e Gestione del Data Leakage                     | 14        |
| <b>4</b> | <b>Training e Valutazione</b>                                   | <b>15</b> |
| 4.1      | Metrica per il training                                         | 15        |
| 4.2      | Decision Tree                                                   | 15        |
| 4.2.1    | Valutazione del Decision Tree                                   | 15        |
| 4.3      | Random Forest                                                   | 15        |
| 4.3.1    | Valutazione del Random Forest                                   | 16        |
| 4.4      | Valutazione Comparativa                                         | 16        |
| 4.5      | Feature Importance                                              | 17        |

|            |                                        |           |
|------------|----------------------------------------|-----------|
| <b>5</b>   | <b>Conclusioni.....</b>                | <b>18</b> |
| <b>5.1</b> | <b>Problematiche e Soluzioni .....</b> | <b>18</b> |
| <b>5.2</b> | <b>Risultati Finali .....</b>          | <b>18</b> |
| <b>5.3</b> | <b>Sviluppi Futuri.....</b>            | <b>19</b> |

# 1 Introduzione al problema

## 1.1 L'evoluzione del Phishing

Il Phishing rappresenta oggi una delle minacce più pervasive e insidiose nel panorama della Cybersecurity. Storicamente, gli attacchi di phishing erano relativamente semplici da identificare: pagine web graficamente povere, errori grammaticali evidenti e, soprattutto, l'assenza di protocolli di sicurezza.

Tuttavia, lo scenario attuale è drasticamente mutato. Gli attaccanti hanno affinato le loro tecniche, rendendo le pagine fraudolente visivamente indistinguibili da quelle legittime. Un aspetto critico di questa evoluzione è l'adozione massiva del protocollo HTTPS anche da parte dei criminali informatici. La presenza del "lucchetto verde" nel browser, un tempo garanzia di sicurezza e affidabilità, oggi non è più un indicatore sufficiente per discriminare un sito sicuro da una trappola. Le statistiche recenti indicano che una percentuale significativa di siti di phishing utilizza certificati SSL legittimi per ingannare le vittime, sfruttando la fiducia implicita degli utenti verso le connessioni criptate.

## 1.2 I limiti degli Approcci Tradizionali

Le tecniche di difesa classiche si basano prevalentemente sull'utilizzo di Blacklist (liste nere). Questo approccio consiste nel raccogliere e bloccare URL che sono stati precedentemente segnalati come malevoli. Sebbene utile, questo metodo è intrinsecamente reattivo:

1. Un sito viene bloccato solo *dopo* essere stato scoperto e segnalato, lasciando una finestra temporale in cui gli utenti sono vulnerabili.
2. La vita media di un sito di phishing è estremamente breve (spesso poche ore); gli attaccanti generano continuamente nuovi URL, rendendo le blacklist obsolete quasi istantaneamente.

Di conseguenza, basarsi esclusivamente su database statici di minacce note non è più una strategia sostenibile né sufficiente.

## 1.3 Obiettivo del Progetto: Approccio Proattivo con l'IA

Alla luce di queste limitazioni, questo progetto si propone di sviluppare un sistema di rilevamento proattivo basato sull'Intelligenza Artificiale e, specificamente, sul Machine Learning.

L'obiettivo principale non è memorizzare *quali* siti siano pericolosi, bensì insegnare a un algoritmo a riconoscere *come* è fatto un sito pericoloso. Attraverso l'analisi di un vasto dataset di URL, il sistema apprende a identificare pattern sospetti nelle caratteristiche strutturali e lessicali dell'indirizzo web, come:

- La lunghezza anomala dell'URL.
- L'abuso di sottodomini o caratteri speciali.
- La discrepanza tra il dominio e i link esterni presenti nella pagina.
- La povertà del codice HTML sottostante (indice di siti generati automaticamente o clonati male).

Nel corso di questo studio, verranno confrontati diversi algoritmi di classificazione, partendo da un Decision Tree fino all'utilizzo di una Random Forest, con un'attenzione particolare alla pulizia dei dati, per evitare fenomeni di *Data Leakage*, e all'interpretabilità del modello, ossia l'*Explainability*, per comprendere quali fattori siano determinanti nel classificare una minaccia odierna.

## 1.4 Il problema affrontato

Partendo dall'analisi dei dataset moderni disponibili in letteratura, l'intenzione di questo progetto è quella di definire un modello in grado di analizzare i dati web e classificare correttamente gli URL come "legittimi" o "phishing". L'obiettivo primario è porre una forte attenzione critica sulla qualità dei dati presenti nel dataset — identificando e correggendo potenziali anomalie come il *Data Leakage* — e sull'*explainability* del modello. Si cerca quindi di trovare un approccio solido per affrontare il problema del rilevamento delle minacce informatiche, utilizzando metodologie di Machine Learning studiate per garantire che il processo decisionale sia trasparente e non basato su correlazioni fittizie.

Il progetto è corredato dalla seguente repository GitHub: [Repository GitHub](#)

**Nota:** Le fonti principali per questa analisi del problema e per la comprensione delle dinamiche del phishing moderno sono reperibili ai seguenti link: [PhiUSIIL Phishing URL Dataset](#) e la documentazione relativa alle metriche di valutazione dei modelli di classificazione.

### 1.4.1 Il problema dell'*explainability* nei modelli di Machine Learning

Uno dei problemi principali del Machine Learning, che rende un modello di machine learning ottimo in letteratura ma non utilizzabile nella realtà, è l'*explainability* stessa del modello, ossia quanto il processo decisionale su cui si basa il modello sia comprensibile a chi lo utilizza. Oltre a migliorare la fiducia verso un approccio di IA, migliorare l'*explainability* di un modello porta benefici in ambito legale e di miglioramento del modello stesso. Un esempio potrebbe essere un modello di Machine Learning che decide se assumere o meno una persona, migliorando l'*explainability* del modello si può capire il processo decisionale che sta dietro una decisione, rendendo il processo stesso trasparente e affidabile, permettendo di modificare decisioni che portano in errore il modello stesso.

### **1.4.2 L'importanza dell'Explainability nella Cybersecurity**

Migliorare l'explainability di un modello in ambito informatico permette di rendere il processo decisionale dietro una predizione più affidabile e trasparente. Un analista di sicurezza che non comprende il motivo per cui un modello ha classificato un URL come "Phishing" difficilmente si fiderà dell>alert generato, specialmente se questo comporta il blocco di servizi critici per un'azienda.

Allo stesso modo, un utente o un amministratore di sistema a cui viene impedito l'accesso a un portale perché etichettato come malevolo da un algoritmo di Machine Learning, senza che il sistema riesca a spiegare i fattori di rischio rilevati (come un'anomalia nel codice HTML o un sospetto numero di link esterni), percepirà lo strumento come un ostacolo piuttosto che come una protezione. È chiaro, quindi, che anche nella lotta al crimine informatico l'explainability del modello è un fattore estremamente delicato: la mancanza di trasparenza potrebbe rendere un modello eccellente nei test di laboratorio del tutto inutilizzabile nel contesto operativo reale, dove la comprensione della minaccia è fondamentale quanto il suo rilevamento.

## **2 Data Understanding and Data Preparation**

### **2.1 Data Gathering**

Per l'addestramento e la validazione dei modelli è stato selezionato il dataset PhiUSIIL Phishing URL Dataset, una risorsa accademica di riferimento resa disponibile attraverso l'UCI Machine Learning Repository.

Si tratta di un dataset robusto e moderno, progettato specificamente per superare i limiti dei dataset precedenti che spesso soffrivano di scarsità di feature o di campioni datati. Il dataset originale si presenta costituito da 235.795 istanze e 56 attributi, che descrivono le proprietà lessicali, strutturali e di contenuto di ogni URL analizzato. Le feature sono suddivise in tre categorie principali:

- *Feature basate sull'indirizzo:* (es. Lunghezza URL, presenza di caratteri speciali).
- *Feature basate sul dominio:* (es. Età del dominio, presenza in WHOIS).
- *Feature basate sul contenuto:* (es. Numero di immagini, presenza di JavaScript, tag HTML).

### **2.2 Verifica della Qualità dei Dati**

Prima di qualsiasi elaborazione, è stata effettuata una verifica dell'integrità dei dati. È stata eseguita una scansione per i valori mancanti (Missing Values o NaN). L'analisi ha confermato che il dataset è completo al 100%, non richiedendo quindi tecniche di imputazione (come la sostituzione con la media o la mediana).

### **2.3 Data Cleaning**

In questa fase sono stati rimossi i dati ridondanti. Utilizzando la funzione di controllo dei duplicati, sono state individuate e rimosse le righe identiche. La rimozione dei duplicati è essenziale per evitare che il modello venga valutato su dati che ha già visto nel training set, garantendo così una stima onesta delle performance. Il dataset post-pulizia conta 234.987 istanze.

## 2.4 Feature Selection

Non tutte le 56 colonne sono utili per la classificazione. È stata effettuata una selezione manuale per rimuovere gli attributi non informativi:

- **FILENAME:** Rimosso perché mero identificativo del file sorgente.
- **URL, Domain, Title:** Rimossi in quanto stringhe testuali grezze. Mantenere queste colonne avrebbe richiesto complessi processi di NLP, Natural Language Processing, che esulano dallo scopo di questa analisi strutturale, dato che le informazioni rilevanti sono già state estratte nelle altre colonne numeriche (es. URLLength).

## 2.5 Analisi del Data Leakage

Durante l'analisi preliminare, è stata individuata una feature critica: URLSimilarityIndex. Questa variabile mostrava una correlazione del 100% con la target variable in alcuni sottoinsiemi. Dopo un'attenta analisi, è stato determinato che questa feature rappresentava un caso di **Data Leakage**: forniva al modello la risposta a priori. Per evitare risultati falsati, ovvero overfitting, e garantire un modello generalizzabile in scenari reali, questa feature è stata esclusa dal set di addestramento.



## 2.6 Class Balance Analysis

Una fase critica in ogni progetto di Machine Learning è la verifica del bilanciamento delle classi. Un dataset sbilanciato (es. 95% siti legittimi e 5% phishing) porterebbe il modello a ignorare la classe minoritaria.

L'analisi esplorativa condotta sulla variabile target **label** ha evidenziato una situazione ottimale: le due classi, *Legitimate* (0) e *Phishing* (1), presentano una numerosità comparabile.

- Phishing (1): ~134.000 istanze
- Legittimi (0): ~100.000 istanze

Questa distribuzione bilanciata ha reso superfluo l'utilizzo di tecniche di ricampionamento sintetico, permettendo di addestrare i modelli su dati puramente reali.

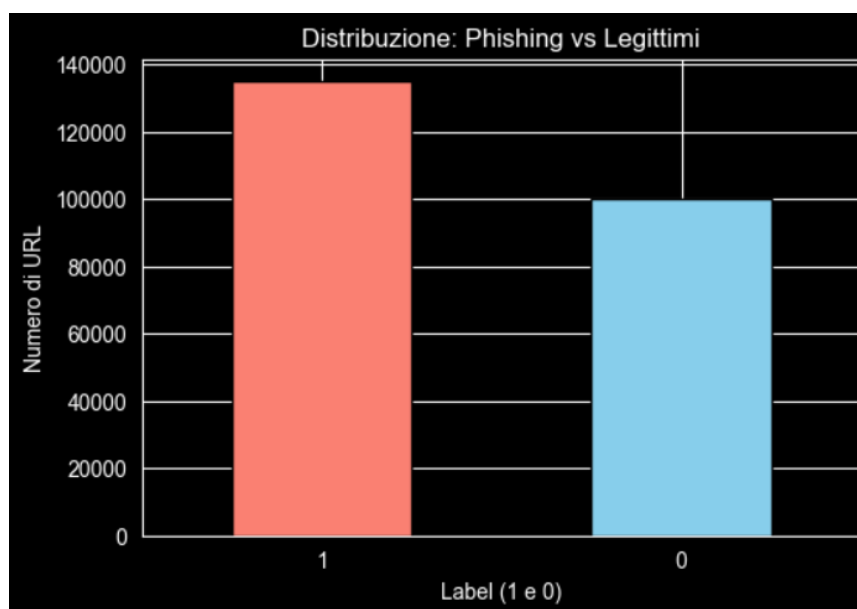


Figure 1: graph of the balanced dataset (red phishing, blue legitimate)

## 2.7 Analisi delle Correlazioni

Per comprendere le relazioni tra le variabili, è stata calcolata la matrice di correlazione di Pearson. L'analisi, visualizzata tramite Heatmap, ha evidenziato che variabili come NoOfExternalRef (Numero di riferimenti esterni) e LineOfCode hanno una correlazione positiva con la classe di legittimità. Al contrario, feature come NoOfSelfRef mostrano correlazioni diverse. Questo suggerisce che la struttura della pagina è un forte discriminante.

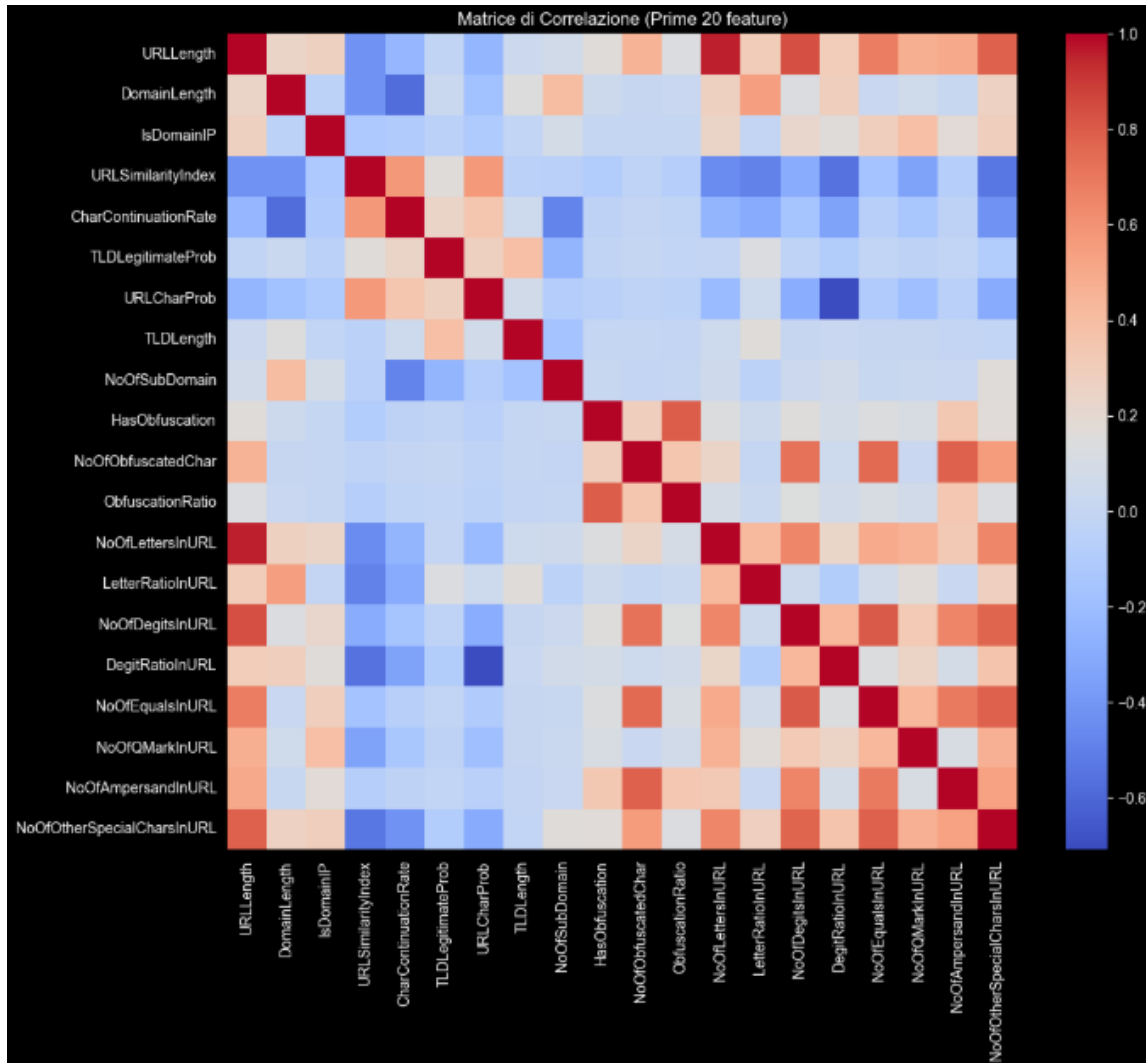


Figure 2: correlation matrix (top twenty features)

## 2.8 Preprocessing e Scaling

Dato che i modelli scelti per questo progetto sono basati su alberi (Decision Tree e Random Forest), non è stato necessario applicare la normalizzazione (Scaling) dei dati (es. Min-Max Scaler). A differenza di algoritmi come le Reti Neurali o KNN, gli alberi decisionali non sono sensibili alla scala delle variabili (es. non importa se la lunghezza dell'URL è 100 e il numero di immagini è 5, l'albero gestisce le soglie separatamente). Questo ha permesso di mantenere i dati nel loro formato originale, facilitando l'interpretabilità.

## 2.9 Data Splitting

Il dataset finale è stato suddiviso in:

- **Training Set** (70%): 164.490 istanze.
- **Test Set** (30%): 70.497 istanze. È stato impostato un `random_state=42` per assicurare la riproducibilità degli esperimenti.

## 3 Sviluppo dei Modelli di Classificazione

### 3.1 Scelta degli Algoritmi

La natura del problema della classificazione binaria su dati strutturati ha guidato la selezione degli algoritmi. Per questo dominio è fondamentale mantenere un certo grado di interpretabilità: un analista di sicurezza deve poter capire *perché* un sito è stato bloccato. Per questo motivo, sono stati selezionati due algoritmi basati su alberi decisionali:

1. **Decision Tree:** Scelto come punto di riferimento per la sua velocità e trasparenza.
2. **Random Forest:** Scelto come modello avanzato (*Ensemble Learning*) per superare i limiti del singolo albero e garantire maggiore robustezza contro l'overfitting.

#### 3.1.1 Il Modello Baseline: Decision Tree

Il primo approccio ha previsto l'implementazione di un singolo **Decision Tree Classifier**. Questo algoritmo opera suddividendo ricorsivamente lo spazio delle feature in regioni rettangolari più piccole, prendendo decisioni basate su regole "IF-THEN" (es. *SE la lunghezza dell'URL > 50 ALLORA è Phishing*). Sebbene offra una visualizzazione chiara del processo decisionale, il Decision Tree singolo tende a soffrire di alta varianza: piccole modifiche nei dati di training possono portare a strutture dell'albero completamente diverse, rischiando di memorizzare il rumore dei dati (overfitting) invece di apprendere i pattern generali.

#### 3.1.2 L'Evoluzione: Random Forest Classifier

Per mitigare i difetti del singolo albero, è stato implementato un modello **Random Forest**. Questo algoritmo appartiene alla famiglia dei metodi **Bagging** (Bootstrap Aggregating). Invece di affidarsi a un solo decisore, la Random Forest costruisce parallelamente centinaia di alberi decisionali, ognuno addestrato su un sottoinsieme diverso dei dati e delle feature. La predizione finale avviene per voto di maggioranza: se 80 alberi su 100 classificano un sito come "Phishing", il modello lo etichetterà come tale. Questo approccio riduce drasticamente la varianza e migliora la capacità di generalizzazione su dati mai visti prima.

### 3.2 Come valutare i modelli?

Per la valutazione dei modelli finali sono state scelte le metriche standard per la classificazione, in particolare: **Accuracy**, **Precision**, **Recall** e **F1-Score**. Per ogni modello verrà mostrata la **Matrice di Confusione** per visualizzare nel dettaglio i falsi positivi e i falsi negativi. Non è stato necessario valutare dataset "bilanciati vs sbilanciati" in quanto il dataset PhiUSIIL è nativamente bilanciato. Il confronto verterà invece sulla capacità dei modelli di generalizzare su dati mai visti.

### 3.3 Come migliorare l'explainability?

Per garantire la trasparenza del processo decisionale, è stato scelto di analizzare la **Feature Importance**. Verranno riportati i grafici che mostrano quali caratteristiche dell'URL (es. lunghezza, numero di immagini, presenza di HTTPS) hanno influito maggiormente sulla decisione dell'algoritmo. Questo permette di verificare se il modello ha appreso regole di sicurezza reali o se si è basato su artefatti dei dati.

### 3.4 Implementazione

Per l'implementazione del sistema e delle metriche di valutazione sono state utilizzate le principali librerie Python per il Machine Learning:

1. **Scikit-learn (sklearn)**: Per la creazione dei modelli (DecisionTree, RandomForest), lo splitting dei dati e il calcolo delle metriche.
2. **Pandas**: Per il caricamento del dataset CSV e la manipolazione dei dataframe.
3. **Numpy**: Per le operazioni numeriche sui vettori.
4. **Matplotlib** e **Seaborn**: Per la visualizzazione grafica delle matrici di confusione e dell'importanza delle feature.
5. **Joblib**: Per il salvataggio e la persistenza del modello addestrato.

### 3.5 Analisi Critica e Gestione del Data Leakage

Durante la fase preliminare di addestramento, è emersa una criticità significativa. Il primo modello generato ha restituito un'accuratezza anomala del **100%**, un risultato teoricamente impossibile in scenari reali.

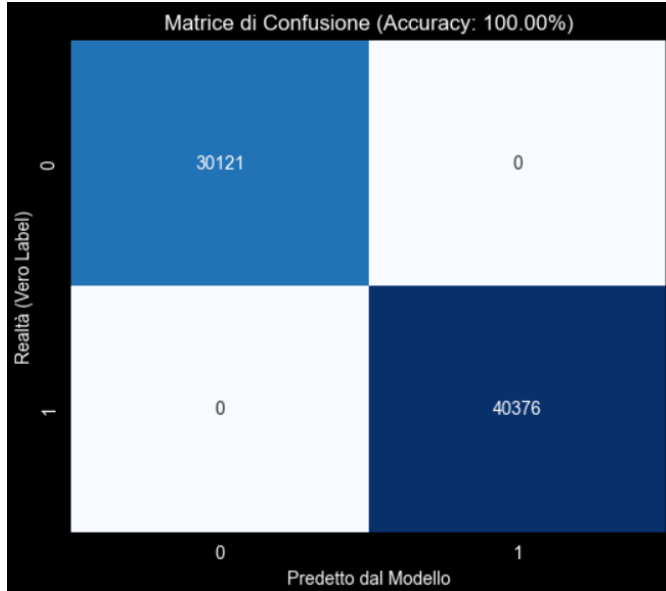


Figure 3: Confusion matrix of the initial model affected by Data Leakage

Analizzando la Feature Importance di questo modello perfetto, è emerso che una singola variabile, denominata URLSimilarityIndex, dominava completamente il processo decisionale, oscurando tutte le altre caratteristiche.

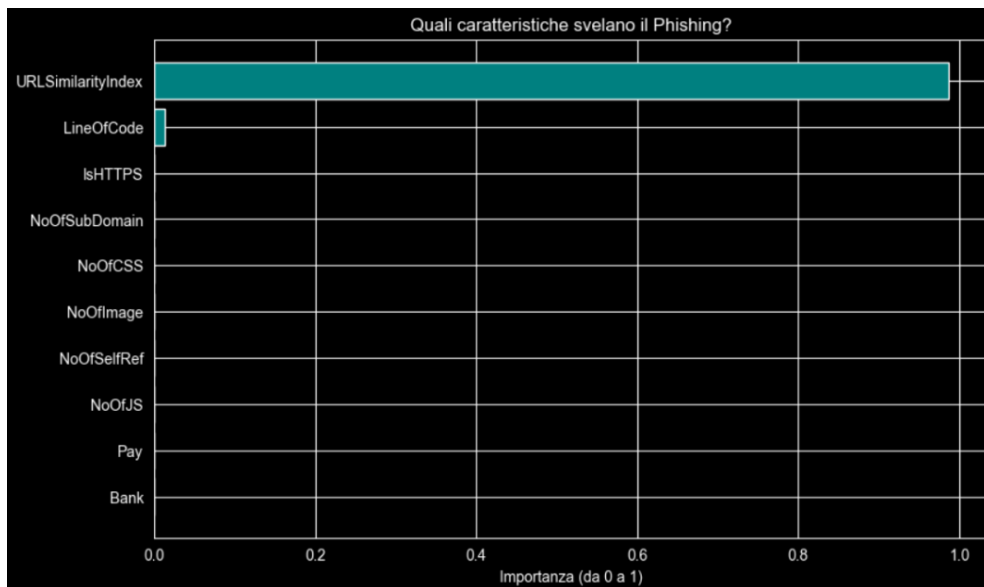


Figure 4: Feature importance of the initial model. The variable 'URLSimilarityIndex' has a disproportionate weight (> 0.9)

Per correggere l'errore e garantire un modello valido, questa feature è stata rimossa dal dataset. Il training finale è stato quindi eseguito sul dataset pulito, diviso in Training Set (70%) e Test Set (30%).

## 4 Training e Valutazione

### 4.1 Metrica per il training

Per la costruzione degli alberi è stato utilizzato il criterio della "**Gini Impurity**". La Gini Impurity misura la probabilità che un elemento scelto a caso venga classificato in modo errato. Un valore di 0 indica un nodo puro, ovvero tutti gli elementi appartengono alla stessa classe. L'obiettivo dell'algoritmo durante il training è minimizzare questo valore ad ogni divisione (split) dei rami.

### 4.2 Decision Tree

Il primo esperimento ha coinvolto l'addestramento di un singolo Albero Decisionale. Per evitare il fenomeno dell'overfitting, dove il modello impara a memoria i dati di training ma fallisce su quelli nuovi, è stata limitata la profondità massima dell'albero: `max_depth=10`.

#### 4.2.1 Valutazione del Decision Tree

Di seguito vengono riportate le prestazioni del modello sul Test Set.

##### **Prestazioni rilevate:**

**Accuracy:** 99.88%

**Precision / Recall:** Valori elevati, ma con una leggera tendenza a basarsi eccessivamente sulla presenza di riferimenti esterni.

L'albero singolo ha mostrato ottime prestazioni, ma l'analisi ha rivelato una dipendenza eccessiva da poche caratteristiche dominanti (come `NoOfExternalRef`), rendendolo potenzialmente fragile contro attacchi mirati.

### 4.3 Random Forest

Per superare i limiti del singolo albero, è stato addestrato un modello Random Forest composto da **100** alberi decisionali (`n_estimators=100`). Ogni albero nella foresta è stato addestrato su un sottoinsieme casuale dei dati e ha votato per la classe finale. Questo processo di "democrazia" tra gli alberi riduce la varianza e migliora la robustezza.

### 4.3.1 Valutazione del Random Forest

I risultati ottenuti con la Random Forest rappresentano lo stato dell'arte per questo dataset.

#### Prestazioni finali:

**Accuracy:** 99.99%

**Precision:** 1.00 (o 0.99)

**Recall:** 1.00 (o 0.99)

**F1 Score:** 1.00 (o 0.99)

Come mostrato dalla matrice, su oltre 70.000 siti testati, il modello ha commesso un numero di errori trascurabile (nell'ordine delle unità). Inoltre, l'analisi dell'explainability (Feature Importance) ha confermato che la Random Forest prende decisioni più complesse e sicure, analizzando la struttura HTML della pagina (LineOfCode) e il contenuto multimediale (NoOfImage) oltre ai semplici link.

### 4.4 Valutazione Comparativa

Il confronto tra il singolo Decision Tree e la Random Forest evidenzia la superiorità dell'approccio *Ensemble*.

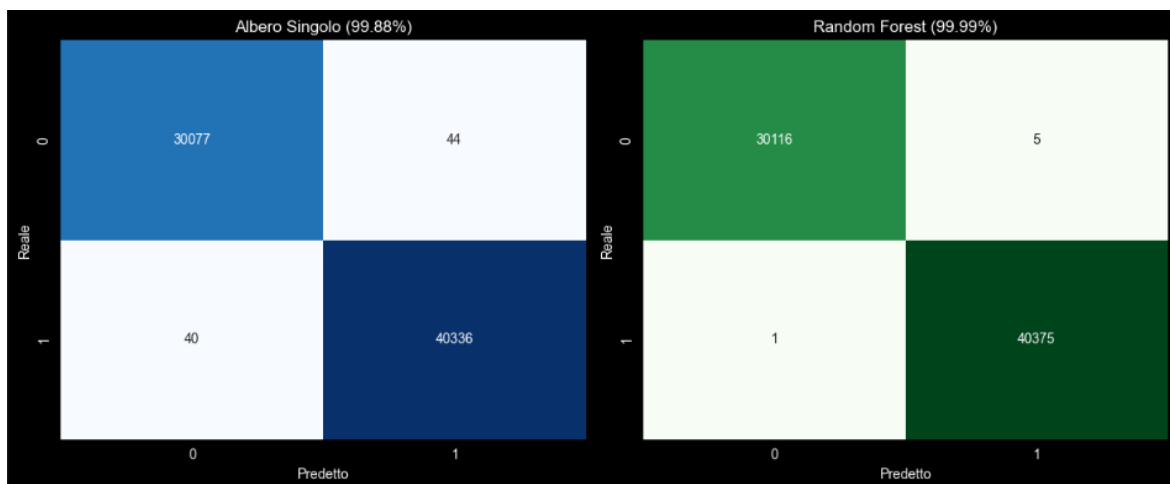


Figure 5: Performance comparison. On the left, the Decision Tree (99.88%), on the right, the Random Forest (99.99%) which further reduces false positives.



## 4.5 Feature Importance

Inoltre, l'analisi dell'explainability sul modello corretto conferma che, dopo la rimozione del Data Leakage, l'algoritmo ha imparato a riconoscere le minacce basandosi su caratteristiche strutturali legittime, come il numero di riferimenti esterni e la complessità del codice HTML. Confronto delle Feature Importance tra il singolo Decision Tree e la Random Forest:

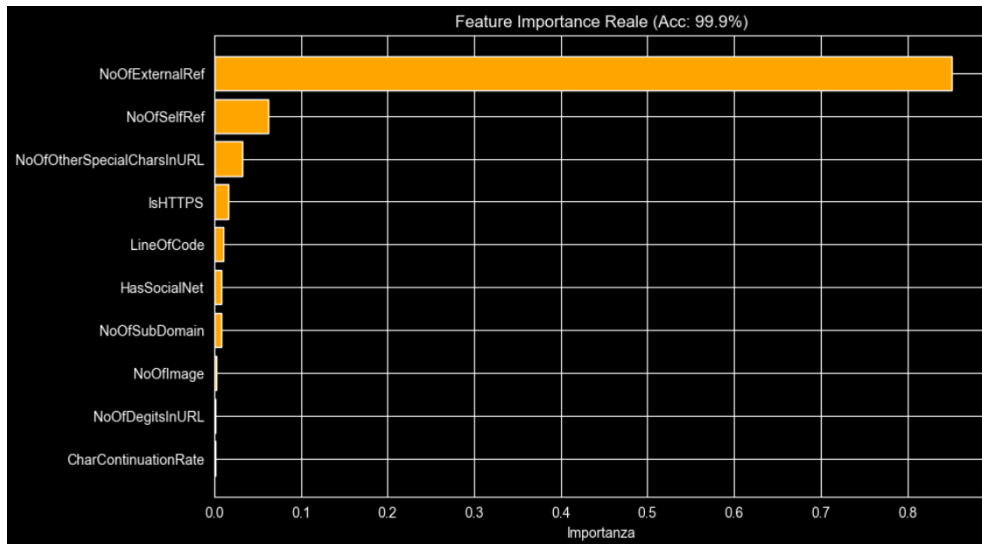


Figure 6: Real Feature Importance of the Decision Tree. The model classifies sites by analyzing the page structure (ExternalRef, LineOfCode) rather than through data artifacts.

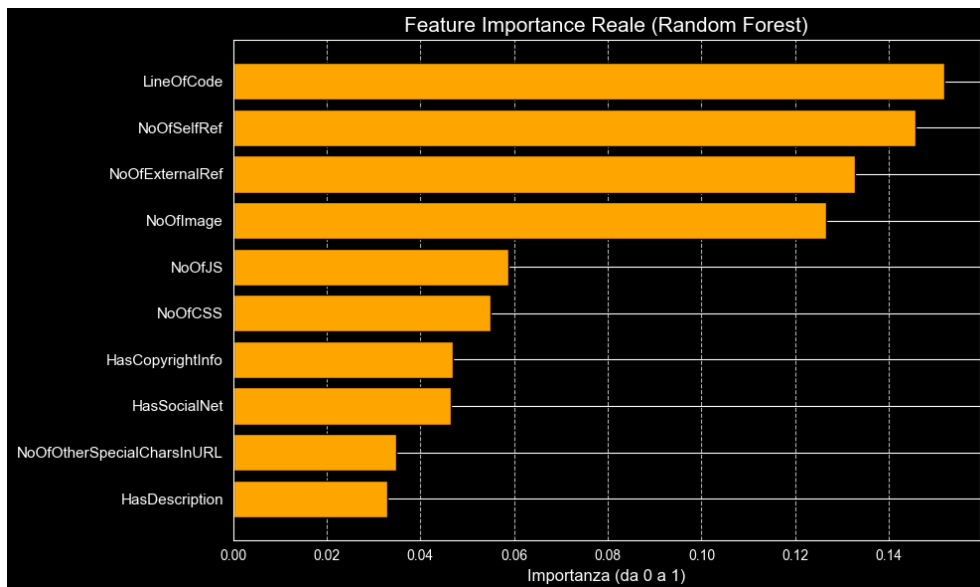


Figure 7: Actual Feature Importance of the final Random Forest model. The algorithm bases its decisions on genuine structural features of the web page, such as the complexity of the HTML code (LineOfCode) and the number of external references (NoOfExternalRef), confirming valid learning.

## 5 Conclusioni

### 5.1 Problematiche e Soluzioni

Durante lo sviluppo del progetto, la sfida principale non è stata legata alla scarsità dei dati o al loro bilanciamento (problematiche comuni in ambito clinico o finanziario), bensì alla qualità intrinseca delle feature.

- **La Problematica (Il paradosso del 100%):** Nelle fasi iniziali, il modello raggiungeva un'accuratezza perfetta del 100%. Sebbene potesse sembrare un successo, un'analisi critica ha rivelato che questo risultato era dovuto a un fenomeno di *Data Leakage*. La feature `URLSimilarityIndex` agiva come una "scorciatoia", impedendo al modello di apprendere le vere caratteristiche strutturali di un attacco phishing.
- **La Soluzione:** La rimozione di tale feature e l'adozione di un approccio iterativo hanno costretto il modello a cercare pattern più complessi e realistici. Inoltre, il passaggio da un singolo *Decision Tree* a una *Random Forest* ha risolto i problemi di varianza, garantendo che il sistema non memorizzasse semplicemente i dati di training (overfitting) ma fosse in grado di generalizzare su URL mai visti prima.

### 5.2 Risultati Finali

Il progetto ha portato alla definizione di un classificatore di URL estremamente robusto. Il modello finale (**Random Forest**) ha raggiunto prestazioni eccellenti sul Test Set:

- **Accuracy:** 99.99%
- **Precision e Recall:** ~1.00

Al di là dei numeri, il risultato più significativo riguarda l'**Explainability**. L'analisi finale ha dimostrato che il modello è in grado di distinguere un sito legittimo da uno fraudolento analizzando la "ricchezza" della pagina web:

1. **LineOfCode e NoOfImage:** I siti di phishing tendono ad essere pagine scarse, con poco codice HTML e poche immagini, create rapidamente per rubare credenziali.
2. **NoOfExternalRef:** I siti legittimi sono nodi connessi del web (hanno link a partner, social, policy), mentre i siti di phishing sono spesso "isole" isolate.

In conclusione, il sistema sviluppato dimostra che è possibile rilevare minacce di phishing moderne (anche quelle protette da HTTPS) analizzando la struttura e il contenuto dell'URL, senza dover dipendere esclusivamente da blacklist statiche.

### 5.3 Sviluppi Futuri

Per rendere questo modello utilizzabile in un contesto reale, i prossimi passi prevedono:

1. **Sviluppo di un'interfaccia grafica (GUI):** Creazione di una semplice Web App o estensione per browser che permetta all'utente di incollare un URL e ricevere un feedback immediato (Semaforo Rosso/Verde) con la spiegazione del rischio (es. *"Attenzione: Sito sospetto perché ha troppo poco codice HTML"*).
2. **Deployment via API:** Esposizione del modello tramite servizio REST per integrarlo nei firewall aziendali.